

USRP Week-1 Lectures Notes

Lecture 1

Professor's Name: Ryan O'Donnell

Course: 15-751 (TCS Toolkit)

Part-1: About the course

Part-2: How to TCS

Part-3: Street Fighting Mathematics

Part-1:

TCS = Theoretical Computer Science

For this course = Algorithms and Computational Complexity (Theory A)

The rest is Theory B (came from volume B of the book)

“STOC/FOCS” topic (Symposium on Theory of Computation/ Foundations of Computer Science)

Toolkit: Intro to mathematical tools and topics that arise in TCS research.

(50% math and 50% CS topics) - probability, complexity theory

Course has more breadth than depth

Background knowledge (helpful to have):

- Reading TCS papers
- Attending TCS talks
- Doing TCS research

Class is for:

- Anyone interested in doing TCS research
- First-year grad student with interest in CS theory
- “Mathematical maturity” is important

Syllabus:

Unit-1: Asymptotics & Probability (Bounds and estimation, Stirling, Binomial coefficients, Central Limit Theorem, Chernoff bounds)

Unit-2: Fourier Transforms (Properties of DFTs, Integer multiplication, Analysis of Boolean functions, Quantum computation)

Unit-3: Algebra & Applications (Number theory, Fields and polynomials, Error correcting codes, Derandomization)

Unit-4: Spectral Graph Theory (Random walks in graphs, Eigenvalues/ eigenvectors, Conductance & mixing, Expander graphs)

Unit-5: CSPs & Hierarchies (LPs and SDPs, Constraint satisfaction, Treewidth, Certification hierarchies)

Unit-6: Information and Learning (Communication complexity, Information theory, Property testing, Learning theory)

Unit-7: Hardness (Cryptography, Alternative assumptions, Average-case hardness, PCP Theorem)

<https://www.diderot.one>

Part-2:

TCS is more or less math. But it's way better. Because you can study whatever you want.

You can study the algorithms and complexity of: Protein folding, Ad auctions, Quantum tomography, Cake cutting, Perverse sheaves, Fundamental computational problems.

Like math, TCS is hard!

Research = sitting around with a pencil & paper, trying to solve that open problem.

Julia Robinson: famed mathematician, proto-TCS researcher

Make 1% progress per day, for 100 days.

Make 10% progress per day, for 10 days.

In the meantime, keep reading, writing, watching, learning, doing math/ TCS!

Read TCS blogs, announcements: <http://cstheory-feed.org>

Follow your favourite researchers on Twitter

Watch videos of talks from elsewhere:

- <https://sites.google.com/site/plustcs/>
- <https://video.ias.edu/csdm>
- <https://simons.berkeley.edu/videos>
- Many conferences (STOC/ FOCS) have videos of the talks available online.
- YouTube channels:
 - CMU Theory, TCS+
 - Princeton TCS
 - Shannon Channel
 - Simons Institute

- Find TCS papers to read (to read one paper it takes almost a month):\
 - Proceedings of recent FOCS/ STOC/ SODA/ CCC
 - Recent posts to arXiv (<https://arxiv.org/archive/cs>)
 - Or ECCC (<https://eccc.weizmann.ac.il>)
 - Papers citing/ citing by the paper you're reading
 - <https://scholar.google.com>

Managing references:

- Save a local copy of all papers you ever look at.
- Use a consistent naming convention.
- Eg. nisan-wigderson-log-rank-conj.pdf
- Maintain a lifetime .bib file for all papers you ever reference.
- Use a consistent BibTex key style. E.g. {NW93}
- Use reference management software: JabRef, BibDesk

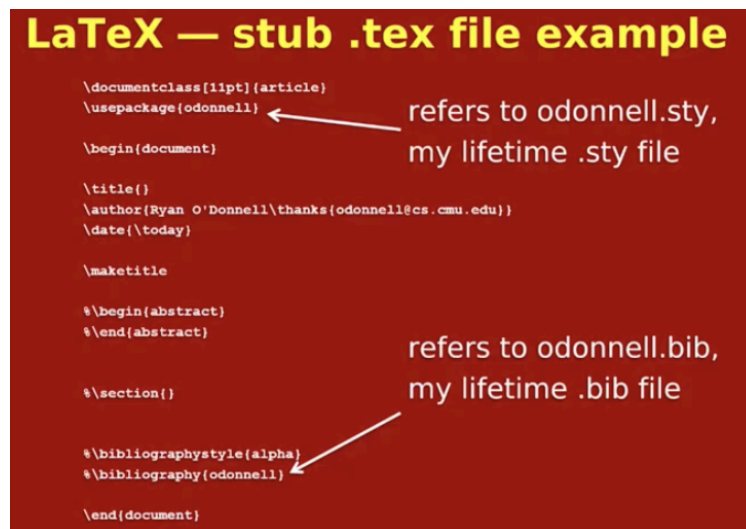
Writing math: LaTeX

- You must become fluent in LaTeX.
- Start with [Overleaf.com](https://overleaf.com)
- Use Google & tex.stackexchange.com to find out how to do anything & everything
- Maybe get "Short Math Guide for LaTeX"

Writing LaTeX is no different from coding. Therefore...

- You need a great text editor:
 - LaTeX syntax highlighting
 - hotkey for PDFLatex-ing, display
 - synchronization/ roundtripping
 - reference autocompletion
 - block commenting...
- Bare minimum: [Overleaf.com](https://overleaf.com)
- Better:
 - WinEdt, TeXnicCenter
 - Texpad, TeXShop
 - Kile
 - Texmaker, TeXstudio
 - Sublime + LaTeXing/ LaTeXTools
 - Emacs + AUCTeX + RefTeX
- You need version-control (even if you're writing a paper solo).
 - Use GitHub
- Use indentation! Make the source code look nice.
 - Every sentence on a new line

Create a lifetime stub .tex file and a lifetime .sty file (for macros, packages).
(And have a lifetime .bib file thanks to your reference management software.)



Set up your future LaTeX workflow.

Make your .bib entries high quality!

- Have a standard for citing proceedings (FOCS, STOC, etc.) and arXiv.
- Get capitalization correct: {B}races needed
- Put in people's first and last names- with the diacritics!
- Use math mode for math parts of titles

Where to get .bib entries:

- Always try www.ams.org/mrlookup first. (The best entries and you don't need ams.org access)
- If that doesn't work, try scholar.google.com (DBLP is also decent.)

LaTeX Do-s:

- $\langle U, V \rangle$
- "quotes"
- $\log(1+x)$
- $\left[\frac{ax+b}{cy}\right]^2$
- If A is a matrix, then
- we execute $\text{normal}\{ALG\}(x)$
- $$\begin{aligned} Y &= (x+1)^2 \\ &= x^2 + 2x + 1 \end{aligned}$$

- Lemma~\ref{lem: big} is (non-breaking space with ~) due to Blum~\cite{Blu99}
- One party, e.g. \ Alice, is (breaking space with \)
- \begin{proof}
 \[
 x=1 \implies x^2=1. \quad \text{\texttt{\textbackslash qedhere}}
 \]
 \end{proof}

Advice:

- This is math, so it has to be 100% correct.
- Take pity on your reader; help them out by formatting well.

Don't be lazy: include figures in LaTeX to help the reader.

- \usepackage{graphicx}
 ...
 \includegraphics{mypicture.png}
- Works with: .jpg, .png, .pdf, .eps

To draw figures, use:

- Inkscape, TikZ, ...
- But there's a big learning curve

Recommendation: draw figures with your presentation software (PowerPoint, Keynote, ...), since you have to learn it anyway...

Download the LaTeX font!

If you write a paper, you'll have to make a talk.

To make a talk, you'll need PowerPoint/ Keynote/ Beamer.

Any of these is fine, but you'll still suffer the "drawing figures" challenge with Beamer.

Learn to integrate beautiful math equations:

- PowerPoint: Built-in Equation Editor is good
 - Alternative: IguanaTex
- Keynote: LaTeXiT/ Equation Editor
- Beamer: Automatic

Presenting math well:

- Kayvon Fatahalian's tips: <http://graphics.stanford.edu/~kayvonf/misc/cleartalktips.pdf>

Part-3:

Street Fighting Mathematics (phrase due to Sanjoy Mahajan - from his book)

If you encounter a problem where you have to predict the next number in the series, do not waste your time on it.

- Just look it up online at oeis.org ([Online Encyclopedia of Integer Sequences](#))

Or if someone asks you what are the Stirling numbers of the second kind, and what the explicit formula for them is, you can look it up on [Wikipedia](#).

Or if you're asked what 0.601907230197 is, you can look it up at [Inverse Symbolic Calculator](#)

And then again if you want to know what the Bessel K function is, look it up on Wikipedia.

If you encounter a problem where you don't know the answer but you do know that someone knows the answer, ask it on mathoverflow.net - you should only ask research-level questions.

Stackexchange sites:

- mathoverflow.net (for research-level questions about math).
- cstheory.stackexchange.com, cs.stackexchange.com (two places for questions about TCS).
- math.stackexchange.com (for help with math questions at any level- except homework)
- tex.stackexchange.com (for any questions about LaTeX)

Or if you want to know something calculus-related, ask Mathematica/ Maple/ Sage.

- Sage is free, and is based around python.
- For quick things, use wolframalpha.com
- You can just enter the question, and it produces the answer for you.

When you're solving an open problem, you should do whatever you can.

Basically, if it's a math problem, and you think someone in history ever thought of using a computer to do it, then Mathematica/ Maple can do it.

- You should also learn Matlab. It's often better for numerical/ matrix things.

Q: Suppose $p(x)$ is a polynomial of degree $\leq n$ which is bounded in $[-1, +1]$ for $x \in [-1, +1]$.
What is the largest $p'(0)$ can be?

Remark: This question actually comes up from time to time in analysis of Boolean functions.

You can *probably* solve it with judicious Googling.

Also appropriate for math.stackexchange.com,
if you put in a reasonable effort first.

Let's solve it using street fighting mathematics.

If you have any problem that involves a number n , you should try small values of n : $n=1$, $n=2$, $n=3$, to try to understand the problem.

“Linear Program” involves maximizing some variable/ linear function subject to some linear inequalities- you can get a computer to solve this.